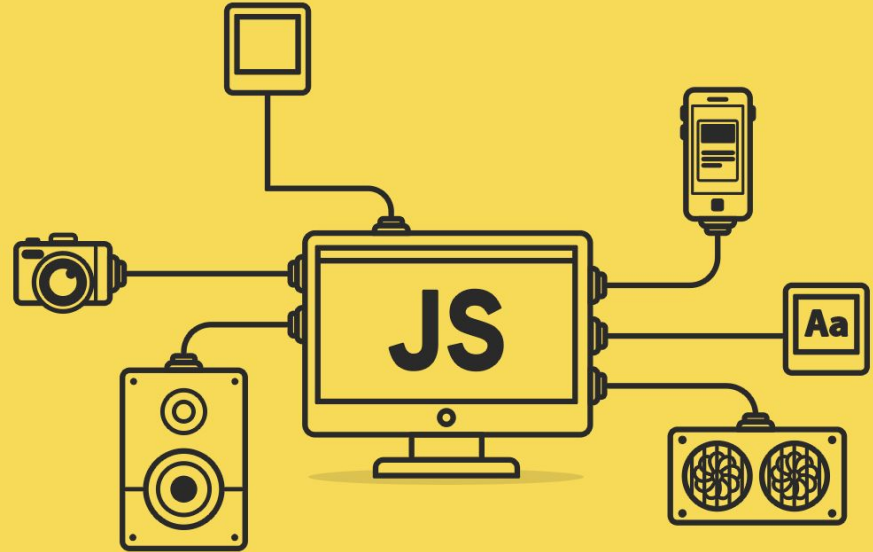


Objetivos da Aula:

- conhecer a linguagem javascript
- criar o ambiente e configurar.
- instalar primeiras bibliotecas.

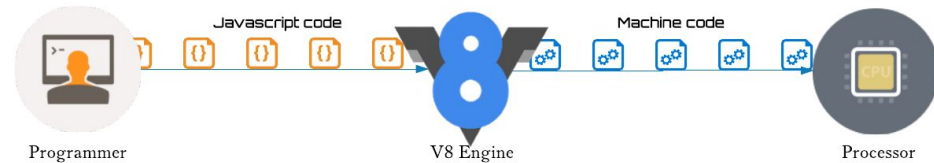
Objetivos da Aula:

O JavaScript começou como uma linguagem para adicionar interatividade aos sites – ele era executado no navegador. Mas, com o tempo, os desenvolvedores perceberam que o JavaScript podia ir muito além do que simplesmente manipular elementos de uma página.

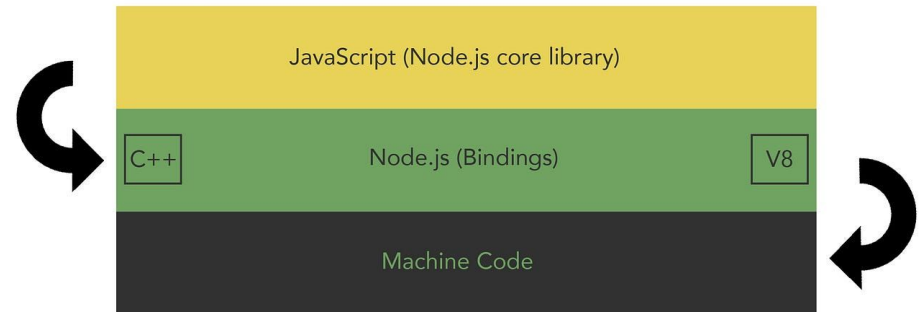


O que é o Node.js?

O Node.js é uma plataforma construída sobre o mecanismo V8, o mesmo motor que o Google Chrome usa para interpretar JavaScript. Mas, ao contrário do JavaScript que conhecemos no navegador, o Node.js roda no servidor.

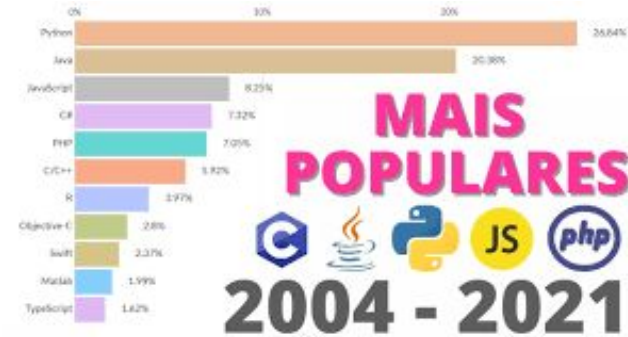


Node.js & Google V8 engine



Causa da Criação

Desde os anos 2000, as aplicações web eram tradicionalmente desenvolvidas usando linguagens diferentes no servidor (como PHP, Java ou Ruby) e JavaScript no cliente. Isso gerava uma certa complexidade. Ryan Dahl, em 2009, lançou o Node.js para aproveitar a eficiência do V8 e resolver problemas como o bloqueio de I/O – ou seja, enquanto um servidor tradicional "espera" por uma operação de leitura ou escrita em disco, ele pode travar ou ocupar uma thread inteira.



Diferenças Fundamentais: Backend vs. Frontend

Frontend:

É a parte da aplicação que roda no navegador. É onde o usuário interage com a interface, os botões, os formulários e tudo o que você vê na tela.

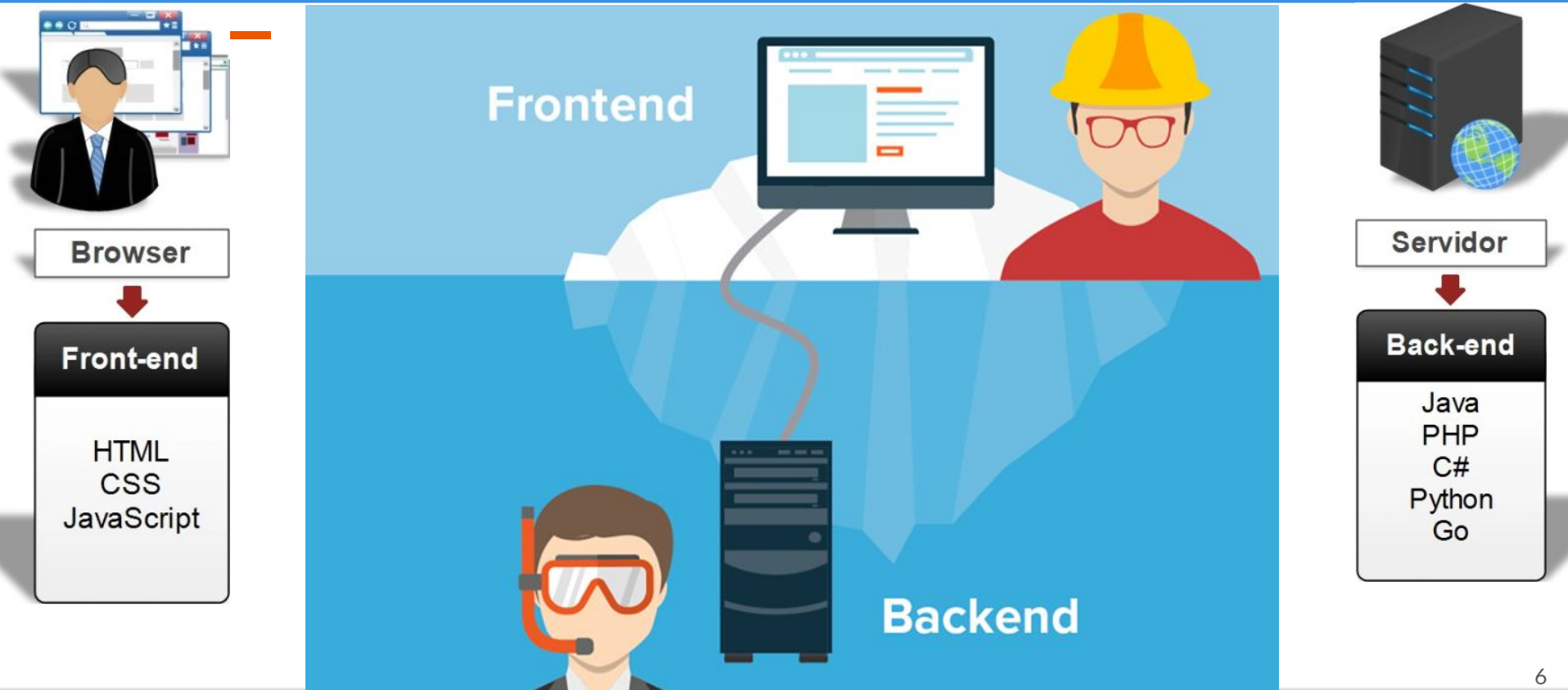
Técnicas como HTML, CSS e JavaScript (rodando no navegador) estão envolvidas no frontend. Em uma arquitetura desacoplada, o frontend consome serviços por meio de APIs, sem ter o conhecimento direto da lógica de negócios.

Backend:

É a "parte de trás" da aplicação, onde toda a lógica de negócios, o acesso ao banco de dados e as regras de segurança são implementadas.

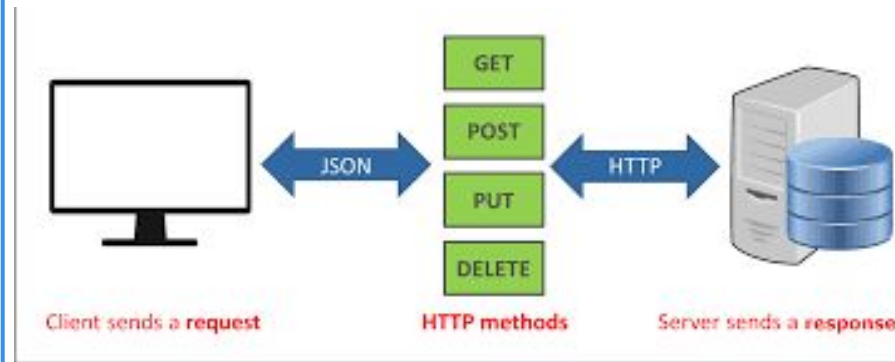
No backend, o Node.js permite construir servidores e APIs que processam as requisições dos clientes e enviam respostas. Aqui, o foco é garantir que os dados sejam tratados, processados e enviados corretamente, de forma rápida e escalável.

Diferenças Fundamentais: Backend vs. Frontend



Arquitetura Desacoplada

Quando dizemos “front e back desacoplados”, estamos falando de separar completamente a interface do usuário (frontend) da lógica de negócio e processamento de dados (backend). O frontend se comunica com o backend por meio de chamadas a APIs. Essa separação permite que o frontend seja mais flexível (por exemplo, podendo ser usado em aplicações mobile, web ou até mesmo desktops)



Por que Usar Node.js no Backend?

Unificação da Linguagem:

Usando JavaScript tanto no front quanto no back, os desenvolvedores podem reaproveitar conceitos e até parte do código. Isso reduz a curva de aprendizado e facilita a comunicação entre equipes.

Desempenho e Escalabilidade:

Graças ao seu modelo de I/O não bloqueante, o Node.js é excelente para aplicações que precisam lidar com muitas conexões simultâneas, como APIs REST e aplicações de chat.



Criando um Servidor Web Simples

1. Instalar o node e verificar a versão.

(<https://nodejs.org/pt-br>)

node -v

npm -v

2. criar um diretório dentro digitar no terminal:

npm init -y

3. criar um arquivo index.js

4. após o código a seguir executar:

node index.js

Arquivo: package.json básico

```
{  
  "name": "aula_01_nodejs",  
  "version": "1.0.0",  
  "description": "aula introdutória",  
  "license": "MIT",  
  "author": "faustinopsy",  
  "type": "module",  
  "main": "index.js",  
  "scripts": {  
    "dev": "node index.js",  
    "test": "echo \\\"Error: no test specified\\\" && exit 1"  
  }  
}
```

O que esse package.json faz?

Ele é um arquivo de configuração geral do projeto, nele temos dentro do objeto scripts : {} os comandos para executar no terminal formas de rodar, buildar “transpilar/compilar”, e ou rodar testes.

e também ele congela as versões de bibliotecas utilizadas, quando ele existe basta rodar o comando no terminal: **npm install** e ele busca no gerenciador de pacotes a bibliotecas que ali estão salvas

Padronizando o package.json

```
{  
  "name": "meu-projeto",  
  "version": "1.0.0",  
  "description": "Projeto de exemplo com Node.js",  
  "main": "index.js",  
  "scripts": {  
    "start": "node index.js",  
    "dev": "nodemon index.js"  
  },  
  "author": "Seu Nome",  
  "license": "MIT",  
  "dependencies": {  
    "express": "^4.18.2"  
  }  
}
```

package.json



package-lock.json



O package-lock.json captura as versões exatas das dependências resolvidas no momento da instalação. Isso garante que todos que usam o projeto tenham uma experiência consistente em diferentes ambientes, o que é crucial para evitar conflitos e garantir a compatibilidade.

Padronizando o package.json

```
{  
  "name": "meu-projeto",  
  "version": "1.0.0",  
  "description": "Projeto de exemplo com Node.js",  
  "type": "module",  
  "main": "index.js",  
  "scripts": {  
    "start": "node index.js",  
    "dev": "nodemon index.js"  
  },  
  "author": "Seu Nome",  
  "license": "MIT",  
  "dependencies": {  
    "express": "^4.18.2"  
  }  
}
```

Instalação de bibliotecas para facilitar o desenvolvimento e produção.

<https://www.npmjs.com/package/nodemon>

```
npm install --save-dev nodemon
```

essa biblioteca faz o servidor ficar rodando continuamente conforme realizamos alterações ele nunca pára e ele mesmo captura as atualizações sem que precisemos parar e iniciar novamente para ver as atualizações

<https://www.npmjs.com/package/express>

essa biblioteca facilita a criação rotas e a captura e extração dos dados da requisição;

```
npm i express
```

<https://www.npmjs.com/package/dotenv>

```
npm install dotenv --save
```

essa biblioteca servirá para escondermos em uma arquivo oculto configurações e senhas e capturar de volta no projeto

Começando a usar o Express

linha 1 importamos a biblioteca
linha 2 adicionamos todas as funcionalidades do express a uma variável, tornando essa variável um objeto com vários métodos internos.

linha 3 intercepta a url e a requisição no primeiro parâmetro **req** e responde com **res**

app.listen: é a inicialização do servidor em uma porta especificada

```
import express from 'express'

const app = express()

app.get('/', (req, res) => {
  res.send('Hello World')
})

app.listen(3000, () => {
  console.log('http://localhost:3000')
})
```

Podemos customizar a porta e o host

Adicionando as linhas

```
import dotenv from 'dotenv'  
  
dotenv.config()  
  
const PORT = process.env.PORT || 8080;  
const HOST = process.env.HOST || '127.0.0.1';
```

aqui chamamos um arquivo de configuração e realizamos a leitura de suas variáveis e caso não exista atribuímos a porta 8080 e o host local

```
import express from 'express'  
import dotenv from 'dotenv'  
dotenv.config()  
  
const app = express()  
  
const PORT = process.env.PORT || 8080;  
const HOST = process.env.HOST || '127.0.0.1';  
  
console.log(process.env.PORT)  
app.get('/', (req, res) => {  
  res.send('Hello World')  
})  
  
app.listen(PORT, HOST, () => {  
  console.log(`Servidor rodando em  
http://${HOST}:${PORT}`);  
})
```

Começando a usar o Express

linha 1 importamos a biblioteca
linha 2 adicionamos todas as funcionalidades do express a uma variável, tornando essa variável um objeto com vários métodos internos.

linha 3 intercepta a url e a requisição no primeiro parâmetro **req** e responde com **res**

app.listen: é a inicialização do servidor em uma porta especificada

```
import express from 'express'

const app = express()

app.get('/', (req, res) => {
  res.send('Hello World')
})

app.listen(3000, () => {
  console.log('http://localhost:3000')
})
```

CommonJS: require e module.exports

Conceitos Fundamentais:

Em CommonJS, usamos a função **require()** para importar módulos e a propriedade `module.exports` (ou simplesmente `exports`) para exportar funcionalidades (funções, objetos, variáveis, etc.).

NÃO IREMOS USAR ESSA FORMA

```
// Importação
const modulo = require('./meuModulo');

// Exportação
module.exports = {
  minhaFuncao,
  outraFuncao,
};
```

CommonJS: require e module.exports

- **Carregamento Síncrono:**

Os módulos são carregados de forma síncrona, o que é adequado para ambientes de servidor, onde o tempo de inicialização geralmente não é tão crítico quanto, por exemplo, em um navegador.

- **Dinamicidade:**

É possível carregar módulos de forma dinâmica, ou seja, o caminho do módulo pode ser determinado em tempo de execução.

Usos Importantes:

- **Ecossistema Node.js:**

A maioria dos módulos nativos e muitos pacotes publicados no npm foram originalmente escritos utilizando o padrão CommonJS.

- **Simplicidade para Scripts:**

Em scripts e pequenos projetos, a abordagem CommonJS é direta e suficiente.

```
// Importação
const modulo = require('./meuModulo');

// Exportação
module.exports = {
  minhaFuncao,
  outraFuncao,
};
```

ES Modules: import e export

Conceitos Fundamentais:

O padrão ES Modules utiliza as palavras-chave **import** e **export** para realizar a importação e exportação de funcionalidades.

```
// Exportação  
export function minhaFuncao() { ... }  
export const minhaVariavel = 42;
```

```
// Importação  
import { minhaFuncao, minhaVariavel } from  
'./meuModulo.js';
```

Referências

PEREIRA, Caio Ribeiro. *Livro de NodeJS*. [S.l.]: Casa do Código, 2013. Atualizado em 09/2021.

PEREIRA, Caio Ribeiro. *Construindo APIs REST com Node.js*. [S.l.]: Casa do Código, 2016. Atualizado em 10/2021.

OSÓRIO, Victor. *Roadmap Back-end: Conhecendo o protocolo HTTP e arquiteturas REST*. [S.l.]: Casa do Código, 2022

PINHO, Diego Martins de. *ECMAScript 6: Entre de cabeça no futuro do JavaScript*. [S.l.]: Casa do Código, 2017. Atualizado em 05/2018.

